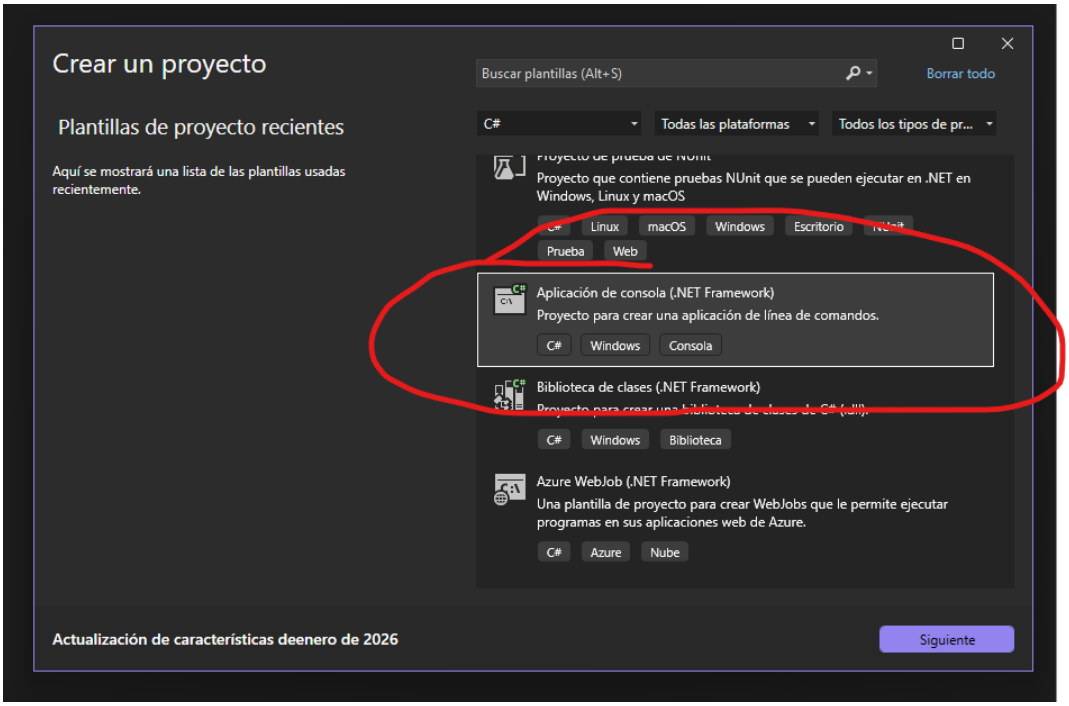
EJERCICIO. METODOS EN C#

El objetivo de este ejercicio es seguir programando con .NET y C# y aprender METODOS

Creamos un proyecto nuevo



Nombre del proyecto METODOS

Vamos a practicar

El profesor explicará los métodos

Void Main (string[] args)

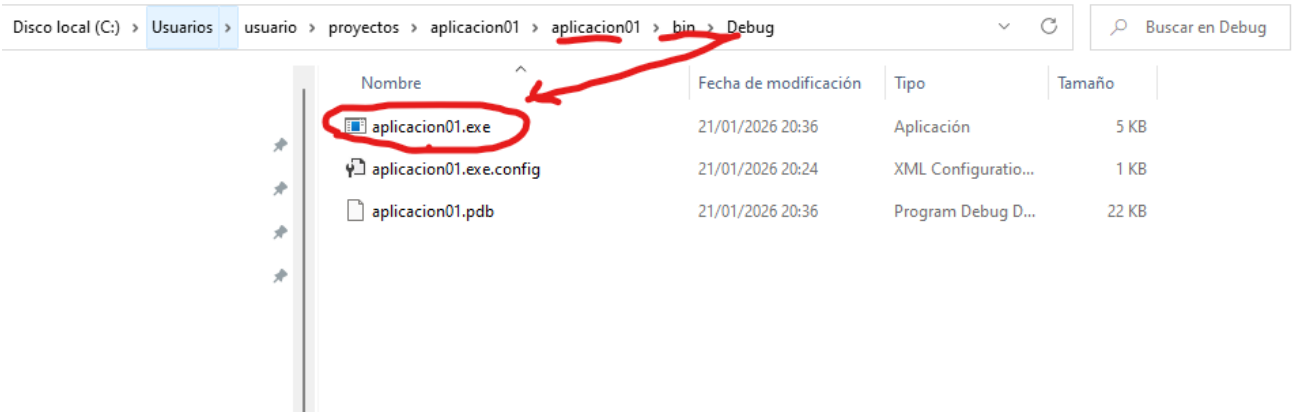
```
namespace aplicacion01
{
    0 referencias
    internal class Program
    {
        // Vamos a crear una aplicación ejecutable en Windows
        // Se le pueden pasar parámetros
        // Hay que ejecutar con start programa.exe param1 param2 ...

        0 referencias
        static void Main(string[] args)
        {
            Console.WriteLine("Ejecución del Programa.");
            //Console.WriteLine(args[0]);

            metodoPrivado();
            Console.WriteLine("Fin del Programa.");
            Console.ReadLine();
        }
        // ninguna otra clase tendrá acceso a este método
        // void significa que no devuelve nada
        1 referencia
        private static void metodoPrivado()
        {
            Console.WriteLine("Esto es el método privado.");
        }
}
```

Vamos a explicar los métodos

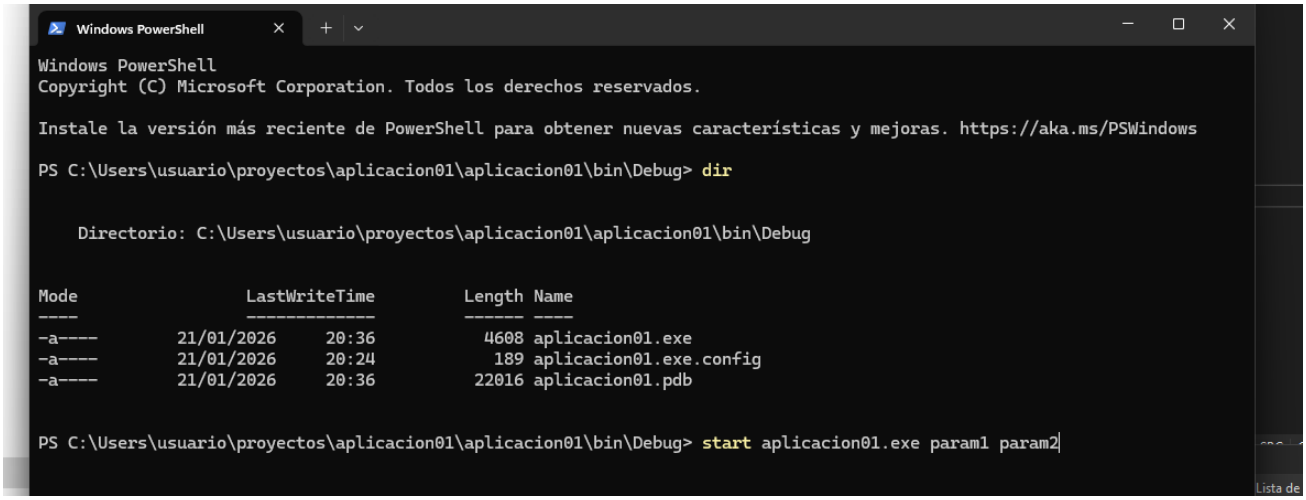
```
0 referencias
static void Main(string[] args)
{
```

Cuando se ejecuta el programa, se define este array de string, para que se puedan ejecutar con parámetros al ser llamada la aplicación

Para ver donde se está generando el programa ejecutable APLICACIÓN.EXE



Abrimos un terminal de Power Shell



Estos 2 parámetros 1 y 2 los podríamos gestionar desde la ejecución de la aplicación

```
1 reference
private void metodo(in int valor)
{
```

El parámetro valor se pasa como referencia, pero solo de lectura.

Es decir, no se puede modificar.

Solo leer y consultar

- También se pueden pasar parámetros con REF
- También se pueden pasar parámetros con OUT

FUNCIONES

```
class Program
{
    0 references
    static void Main()
    {
        int data;
        new Program().metodo(out data);
        Console.WriteLine(data);
        Console.ReadLine();
    }
    1 reference
    private void metodo(out int valor)
    {
        valor = funcion();

        static int funcion() => 50 + 20;

    }
}
```

Aquí estamos viendo la demo de una funcion lambda

static int funcion() => 50 + 20

Hay que tener claro el tema de las variables locales y globales. Los métodos estáticos se relacionan con las variables globales.

Módulo 1 – 10 ejercicios fáciles de métodos (con soluciones)

Objetivo: practicar definición y uso de métodos en C# (parámetros, retorno y sobrecarga básica).

Sugerencia: resuelve cada ejercicio creando un método y probándolo desde Main().

Ejercicio 1 – Saludo

Enunciado: Crea un método void llamado Saludar que reciba un nombre y escriba: "Hola, <nombre>".

Solución:

```
static void Saludar(string nombre)
{
    Console.WriteLine($"Hola, {nombre}");
}

// Prueba:
Saludar("Ana");
```

Ejercicio 2 – Suma de dos números

Enunciado: Crea un método int Sumar(int a, int b) que devuelva la suma.

Solución:

```
static int Sumar(int a, int b)
{
    return a + b;
}

// Prueba:
Console.WriteLine(Sumar(3, 4));
```

Ejercicio 3 – Es par

Enunciado: Crea un método bool EsPar(int n) que devuelva true si n es par.

Solución:

```
static bool EsPar(int n)
{
    return n % 2 == 0;
}

// Prueba:
Console.WriteLine(EsPar(10));
```

Ejercicio 4 – Mayor de dos

Enunciado: Crea un método int Mayor(int a, int b) que devuelva el mayor de los dos.

Solución:

```
static int Mayor(int a, int b)
{
    return (a > b) ? a : b;
}

// Prueba:
Console.WriteLine(Mayor(7, 2));
```

Ejercicio 5 – Convertir a mayúsculas

Enunciado: Crea un método string AMayusculas(string texto) que devuelva el texto en mayúsculas.

Solución:

```
static string AMayusculas(string texto)
{
    return texto.ToUpper();
}

// Prueba:
Console.WriteLine(AMayusculas("hola"));
```

Ejercicio 6 – Contar letras

Enunciado: Crea un método int ContarCaracteres(string texto) que devuelva cuántos caracteres tiene.

Solución:

```
static int ContarCaracteres(string texto)
{
    return texto.Length;
}

// Prueba:
Console.WriteLine(ContarCaracteres("C#"));
```

Ejercicio 7 – Área de un rectángulo

Enunciado: Crea un método double AreaRectangulo(double baseR, double altura) que devuelva el área.

Solución:

```
static double AreaRectangulo(double baseR, double altura)
{
    return baseR * altura;
}

// Prueba:
Console.WriteLine(AreaRectangulo(5, 2));
```

Ejercicio 8 – Repetir texto (parámetro opcional)

Enunciado: Crea un método void Repetir(string texto, int veces = 1) que imprima el texto "veces" veces.

Solución:

```
static void Repetir(string texto, int veces = 1)
{
    for (int i = 0; i < veces; i++)
        Console.WriteLine(texto);
}

// Prueba:
Repetir("Hola");
Repetir("Hola", 3);
```

Ejercicio 9 – Sobrecarga de Sumar

Enunciado: Crea 2 métodos Sumar: uno que sume 2 int y otro que sume 3 int.

Solución:

```
static int Sumar(int a, int b) => a + b;
static int Sumar(int a, int b, int c) => a + b + c;

// Prueba:
Console.WriteLine(Sumar(1, 2));
Console.WriteLine(Sumar(1, 2, 3));
```

Ejercicio 10 – Intentar convertir texto a número

Enunciado: Crea un método bool IntentarConvertir(string texto, out int numero) que use int.TryParse y devuelva si se pudo.

Solución:

```
static bool IntentarConvertir(string texto, out int numero)
{
    return int.TryParse(texto, out numero);
}

// Prueba:
if (IntentarConvertir("123", out int n))
    Console.WriteLine(n);
else
    Console.WriteLine("No es número");
```

Por último te propongo un ejercicio más difícil

Enunciado: Mini gestor de notas (menú + validaciones)

Crea una aplicación de consola que permita gestionar notas numéricas (0 a 10) usando métodos. El programa debe mostrar un menú en bucle hasta que el usuario elija salir.

Menú

1. 1. Añadir nota
2. 2. Mostrar todas las notas
3. 3. Mostrar estadísticas (media, máxima, mínima, aprobados)
4. 4. Buscar una nota (¿existe exactamente?)
5. 0. Salir

Reglas

- La colección principal será: List<double> notas.
- Una nota es válida si está entre 0 y 10 (incluidos).
- Debes usar métodos para organizar el programa (no todo en Main).
- Valida la entrada: si el usuario escribe algo no numérico o fuera de rango, se vuelve a pedir.

Métodos mínimos sugeridos

- static void MostrarMenu()
- static int LeerOpcionMenu()
- static double LeerNotaValida()
- static void AnadirNota(List<double> notas)

- static void MostrarNotas(List<double> notas)
- static void MostrarEstadisticas(List<double> notas)
- static void BuscarNota(List<double> notas)
- static bool ExisteNota(List<double> notas, double notaBuscada)

Solución propuesta (ejemplo completo)

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        var notas = new List<double>();
        int opcion;

        do
        {
            MostrarMenu();
            opcion = LeerOpcionMenu();

            switch (opcion)
            {
                case 1:
                    AnadirNota(notas);
                    break;
                case 2:
                    MostrarNotas(notas);
                    break;
                case 3:
                    MostrarEstadisticas(notas);
                    break;
                case 4:
                    BuscarNota(notas);
                    break;
                case 0:
                    Console.WriteLine("Saliendo...");
                    break;
                default:
                    Console.WriteLine("Opción no válida.");
                    break;
            }

            Console.WriteLine();
        }
        while (opcion != 0);
    }

    static void MostrarMenu()
    {
        Console.WriteLine("=== MINI GESTOR DE NOTAS ===");
```

```
    Console.WriteLine("1. Añadir nota");
    Console.WriteLine("2. Mostrar todas las notas");
    Console.WriteLine("3. Mostrar estadísticas");
    Console.WriteLine("4. Buscar una nota");
    Console.WriteLine("0. Salir");
    Console.Write("Elige una opción: ");
}

static int LeerOpcionMenu()
{
    while (true)
    {
        string? input = Console.ReadLine();
        if (int.TryParse(input, out int opcion))
            return opcion;

        Console.Write("Entrada inválida. Elige una opción (número): ");
    }
}

static double LeerNotaValida()
{
    while (true)
    {
        Console.Write("Introduce una nota (0 a 10): ");
        string? input = Console.ReadLine();

        if (double.TryParse(input, out double nota) && nota >= 0 && nota <= 10)
            return nota;

        Console.WriteLine("Nota inválida. Debe ser un número entre 0 y 10.");
    }
}

static void AnadirNota(List<double> notas)
{
    double nota = LeerNotaValida();
    notas.Add(nota);
    Console.WriteLine($"Nota {nota} añadida correctamente.");
}

static void MostrarNotas(List<double> notas)
{
    if (notas.Count == 0)
    {
        Console.WriteLine("No hay notas registradas.");
        return;
    }

    Console.WriteLine("Notas registradas:");
    for (int i = 0; i < notas.Count; i++)
```

```
        {
            Console.WriteLine($"{i + 1}. {notas[i]}");
        }
    }

    static void MostrarEstadisticas(List<double> notas)
    {
        if (notas.Count == 0)
        {
            Console.WriteLine("No hay notas para calcular estadísticas.");
            return;
        }

        double suma = 0;
        double max = notas[0];
        double min = notas[0];
        int aprobados = 0;

        foreach (var n in notas)
        {
            suma += n;
            if (n > max) max = n;
            if (n < min) min = n;
            if (n >= 5) aprobados++;
        }

        double media = suma / notas.Count;

        Console.WriteLine($"Media: {media:F2}");
        Console.WriteLine($"Máxima: {max}");
        Console.WriteLine($"Mínima: {min}");
        Console.WriteLine($"Aprobados (>=5): {aprobados} de {notas.Count}");
    }

    static void BuscarNota(List<double> notas)
    {
        if (notas.Count == 0)
        {
            Console.WriteLine("No hay notas registradas para buscar.");
            return;
        }

        double buscada = LeerNotaValida();
        bool existe = ExisteNota(notas, buscada);

        Console.WriteLine(existe ? "La nota existe en la lista." : "La nota NO existe en
la lista.");
    }

    static bool ExisteNota(List<double> notas, double notaBuscada)
    {
```

```
        return notas.Contains(notaBuscada);
    }
}
```

Mas propuestas si te apetece programar más horas.

Puedes ampliarlo pidiendo al usuario confirmación antes de borrar notas

Ampliar guardando las notas en disco (TXT)

Ampliar guardando las notas en disco (EXCEL .XLSX)